WEEK 3 / LESSON 5 / 50 MINUTES

L5

Backend with Node.js

You call Paytm. HDFC. Ola. UPI. Flipkart. Zerodha. Every screen in your life talks to an API. Today you build one — not a toy, a production-shaped skeleton you will extend in Project 2.

When did you last think about the API call behind your favourite app? The best ones disappear.

WHAT'S INSIDE

WHY THIS LESSON MATTERS

A good API is boring. Boring is a compliment.

WHY

Real

A real reason this matters in industry today.

SCALE

Today

A real reason this matters in industry today.

SPEED

Now

A real reason this matters in industry today.

IMPACT

Yours

A real reason this matters in industry today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1. Describe the request-response lifecycle of a REST API.
2. Build a 3-endpoint Express server with Zod validation.
3. Explain the 6 REST constraints practically.
4. Use HTTP status codes the right way.
5. Containerise the API with one Dockerfile.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

Request-response lifecycle

4 steps, every call

02

Express in 25 lines

3 endpoints, validated, production-shaped

03

The 6 REST constraints —...

In daily work, only Stateless and Uniform Interface matter. The others follow.

04

Status codes — the contract

Status codes ARE the contract. Pick them intentionally.

01

PART 1 OF 4

Request-response lifecycle

4 steps, every call

CONCEPT 1 OF 4

Request-response lifecycle

Client → HTTP request.
Router parses verb + path.
Middleware runs (auth, validation, logging).
Handler returns response with a status code.
SHAPE
Client → MW → Handler

Master this cycle; every framework then makes sense.

DEFINITION

Request-response lifecycle

4 steps, every call

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 2 OF 4

Express in 25 lines

app.get("/health", …) → {ok: true}
app.get("/schema", …) → returns input JSON schema
app.post("/predict", zod.safeParse → price)
app.listen(3000, "running")
STARTER
25 lines

Type along. This skeleton is the basis of P2.

KEY POINTS

- app.get("/health", …) → {ok: true}
- app.get("/schema", …) → returns input JSON schema
- app.post("/predict", zod.safeParse → price)
- app.listen(3000, "running")

CONCEPT 3 OF 4

The 6 REST constraints — practical view

In daily work, only Stateless and Uniform Interface matter. The others follow.

DEFINITION

The 6 REST constraints — practical view

In daily work, only Stateless and Uniform Interface matter. The others follow.

EXAMPLE

Real-world example: applies directly to Indian industry contexts.

CONCEPT 4 OF 4

Status codes — the contract

Status codes ARE the contract. Pick them intentionally.

KEY POINTS

- Status codes ARE the contract. Pick them intentionally
- Practical, named, applied to a real Indian use-case.
- Practical, named, applied to a real Indian use-case.

02

PART 2 OF 4

Engage and reflect

Pause, talk, and connect what you just learned to your own work.

✦ THINK · PAIR · SHARE

ACTIVITY

Break your own API

PROMPT

Apply the four concepts above to one real example from your own week.

HOW TO RUN IT

1.

2.

3.

4. Reveal: instructor confirms the canonical answer.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1 *Which of the 6 REST constraints do you think modern teams honour least?*

Q2 *What status code do you confuse most — and why?*

Q3 *Would you onboard a new engineer onto your most recent project in under 3 minutes?*

03

PART 3 OF 4

Knowledge check

Three short questions. One minute each. Honest answers only.

KNOWLEDGE CHECK · QUESTION 1 OF 3

POST created a user. Status code?

A B

B D

C Answer: B. 201 Created for successful resource creation.

D (no option)

ANSWER · QUESTION 1 OF 3

POST created a user. Status code?

B

CORRECT ANSWER

D

WHY

The correct option matches the lesson's core principle.

KNOWLEDGE CHECK · QUESTION 2 OF 3

Validation failed. Status code?

A B

B D

C Answer: C. 400 Bad Request with a machine-readable error body.

D (no option)

ANSWER · QUESTION 2 OF 3

Validation failed. Status code?

B

CORRECT ANSWER

D

WHY

The correct option matches the lesson's core principle.

KNOWLEDGE CHECK · QUESTION 3 OF 3

"Stateless" means:

A No DB

B No session on the server

C No cookies

D No cache

ANSWER · QUESTION 3 OF 3

"Stateless" means:

B

CORRECT ANSWER

No session on the server

WHY

Each request carries all the context needed; no server-side session

04

PART 4 OF 4

Apply and close

One thing to remember. One thing to ship this week.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

1. Pick the right tool — never the loudest one.
2. Practice the framework on a real example.
3. Watch for the three classic pitfalls.
4. Document your decision in one sentence.
5. Carry the discipline forward to the next lesson.

WORKED EXAMPLE

How this lesson plays out in one real Indian context

THE SCENARIO

Client → HTTP request.
Router parses verb + path.
Middleware runs (auth, validation, logging).
Handler returns response with a status code.
SHAPE
Client → MW → Handler

Master this cycle; every framework then makes sense.

THE TAKEAWAY

"A good API is boring. Predictable. Documented. Fast. That boredom is what production teams pay for."

WHAT TO NOTICE

1. Request-response lifecycle
2. Express in 25 lines
3. The 6 REST constraints —...
4. Status codes — the contract

APPLY IT → ASSIGNMENT

Mini assignment — add /metrics

Extend the 25-line Express server with a /metrics endpoint that returns request count + median latency since server start.

DELIVERABLES

- In-memory counters OK (no DB required).
- Return JSON with {requests, median_latency_ms}.
- Make /metrics idempotent.
- Push to GitHub.

WHAT 'GOOD' LOOKS LIKE

Deliverable: Repo URL + curl output of /metrics on the forum.

ONE THING TO REMEMBER

> *"A good API is boring. Predictable. Documented. Fast. That boredom is what production teams pay for."*

— AI Learning Hub

END OF LESSON 3

What you can now do

Each one of these is now part of your working vocabulary.

1. Pick the right tool — never the loudest one.
2. Practice the framework on a real example.
3. Watch for the three classic pitfalls.
4. Document your decision in one sentence.
5. Carry the discipline forward to the next lesson.

NEXT → Lesson 6 — Database Design. SQL vs NoSQL for AI.